

SOFTWARE ENGINEERING



Group Members:

- Mian Abubakar (01-134251-043)
- Mirza Uzair Baig (01-134251-044)
- Abdul Rehman Khan (01-134251-
 - Qasver Ali (01-134251-061)

Smart Railway Scheduling System

➤ Introduction:

A *Smart Railway Scheduling System* (SRSS) is a software system designed to manage and optimize railway train schedules. It helps railway authorities plan train routes, departure and arrival times, platform allocation, track usage, and train movements.

In a traditional railway system, scheduling may require significant manual coordination between railway operators, station staff, and train controllers. A smart scheduling system can automate many of these activities and update schedules when unexpected situations such as train delays, track maintenance, or platform availability occur.

The system maintains information about trains, stations, routes, tracks, platforms, schedules, and passengers. The purpose of this project is to analyze and document the requirements, models, architecture, design, and testing approach of such a system.

➤ Problem Statement:

Managing railway schedules manually can become difficult when many trains operate on different routes and share the same tracks and platforms. A delay in one train can affect other trains using the same track or platform. Similarly, maintenance work or an unavailable platform may require the railway schedule to be changed.

The proposed *Smart Railway Scheduling System* will provide a centralized system for managing railway schedules. It will allow authorized railway staff to create schedules, allocate platforms and tracks, monitor train status, and modify schedules when required.

➤ **Project Scope:**

Included In the System:

1. Train registration and management, Station management, route management, Track management, platform management.
2. Train scheduling, Train departure and arrival information
3. Platform allocation, Track allocation
4. Delay management, Schedule modification
5. Train status monitoring, Passenger schedule searching
6. Notifications about schedule changes
7. User authentication
8. Reports for railway operators

Outside the Scope:

The following activities are not directly included:

1. Physical operations of trains or controlling train engines
2. Railway ticket payment processing
3. Physical maintenance of Railway tracks
4. Automatic operations of Railway signals
5. Manufacturing or repairing trains

➤ **Functional Requirements:**

1. User Management:

The administrator shall be able to create, update, deactivate, and manage user accounts.

2. Train Management:

The system shall allow authorized staff to add, update, and remove train information. Train information may include:

- Train ID
- Train Name
- Train Type
- Capacity
- Current Status

3. Station management:

The system should maintain information about railway stations. Each station would have:

- Station ID
- Station Name
- Location
- Platforms
- Available Tracks

4. Route management:

The system shall allow railway staff to create and manage routes between stations.

5. Schedule Creation:

The scheduler shall be able to create a schedule for a train by specifying:

- Train
- Route
- Departure time
- Arrival time
- Stations
- Platform

6. Platform Allocation:

The system shall allocate available platforms to trains according to their schedules.

7. Conflict Detection:

The system shall detect scheduling conflicts when two trains are assigned the same track or platform at overlapping times.

8. Train Status:

The system shall maintain train statuses such as:

- Scheduled
- Boarding
- Departed
- Delayed
- Arrived
- Cancelled

9. Schedule Display:

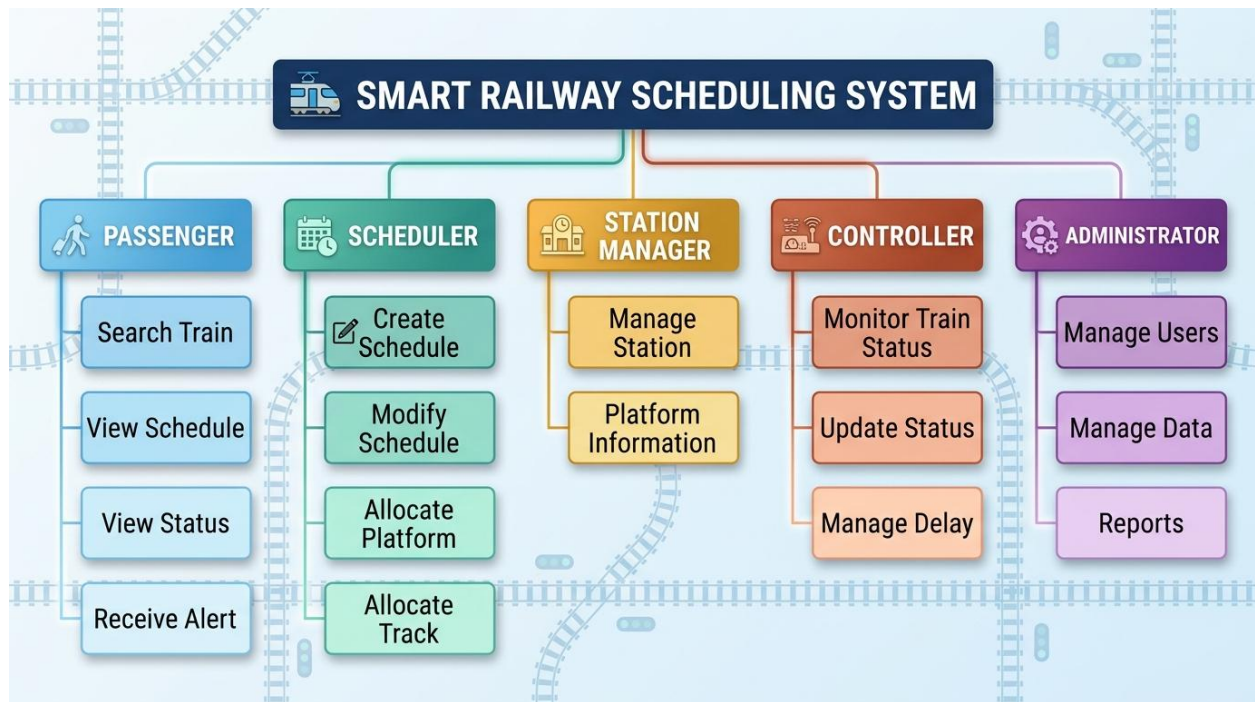
The system shall display:

- Train number
- Train name
- Departure time
- Arrival time
- Platform
- Status

➤ Non-Functional Requirements:

Requirements	Description
Performance	The system should respond quickly to schedule searches and updates.
Reliability	The system should remain available during railway operating hours.
Security	Only authorized users should be able to modify schedules.
Availability	The system should be available whenever railway operations require it.
Scalability	The system should support additional trains and stations in the future.
Maintainability	The software should be easy to maintain and update.
Accuracy	Schedule, platform, and track information should be accurate.

➤ Use Case Diagram:



➤ Use Case Description Tables:

1. Search Train:

Field	Description
Use case	Search train
Actor	Passenger
Purpose	Find a train between two stations
Input	Departure, destination and date
Main Flow	Passengers enter search information → system searches schedules → System displays available trains
Alternative Flow	No train is available → system displays “No train found”

2. Create Schedule:

Field	Description
Use case	Create schedule
Actor	Railway scheduler
Purpose	Create a new train schedule
Input	Train, route, time, platform and track
Main Flow	Scheduler selects trains → selects route → enters time → system checks conflicts → schedule is saved
Alternative Flow	Conflict detected → system asks scheduler to select another time/platform/track

3. Modify Schedule:

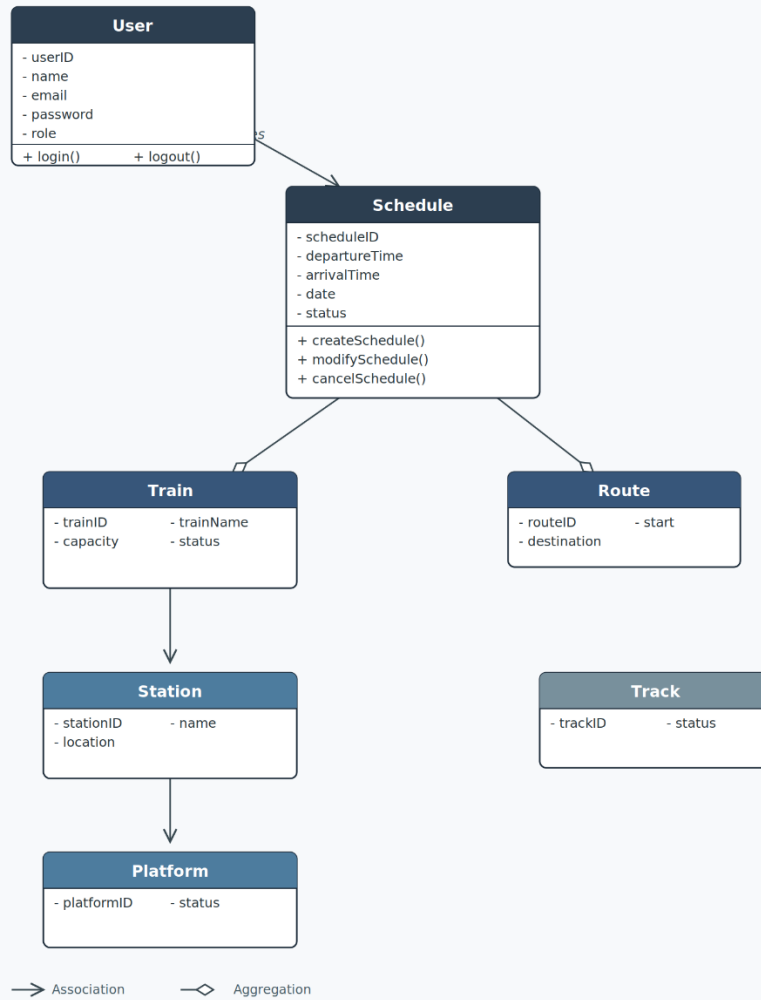
<i>Field</i>	Description
Use Case	Modify Schedule
Actor	Railway Scheduler
Purpose	Change an existing schedule
Input	Existing schedule and new information
Main Flow	Scheduler selects schedule → changes information → system checks conflicts → system saves changes
Alternative Flow	Conflict occurs → system rejects change and displays warning

4. Update Train Status:

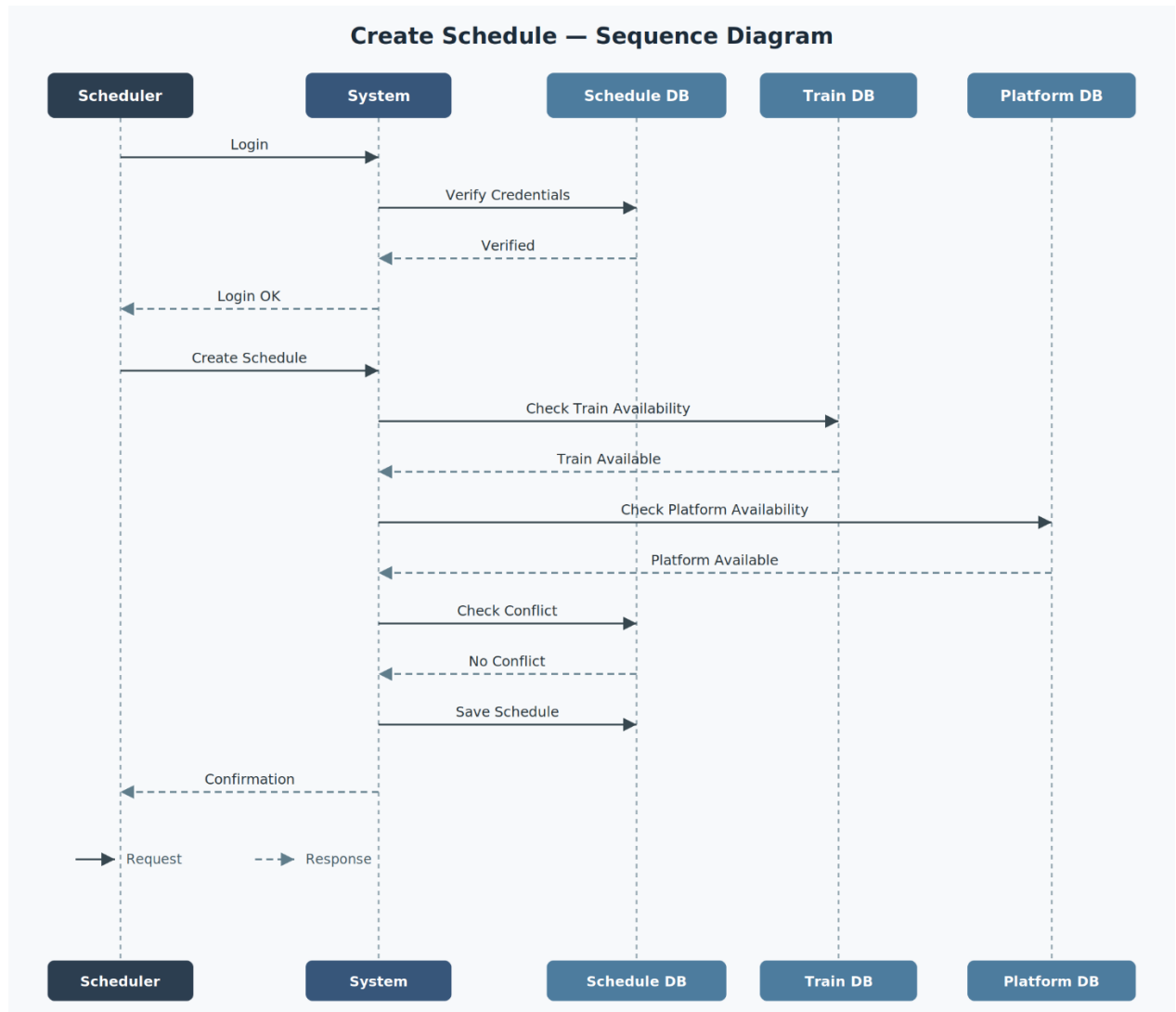
<i>Field</i>	Description
<i>Use case</i>	Update Train Status
<i>Actor</i>	Train Controller
<i>Purpose</i>	Update the current status of a train
<i>Input</i>	Train ID and new status
<i>Main Flow</i>	Controller selects train → selects status → system updates status
<i>Alternative Flow</i>	Invalid train ID → system displays error

➤ Class Diagram:

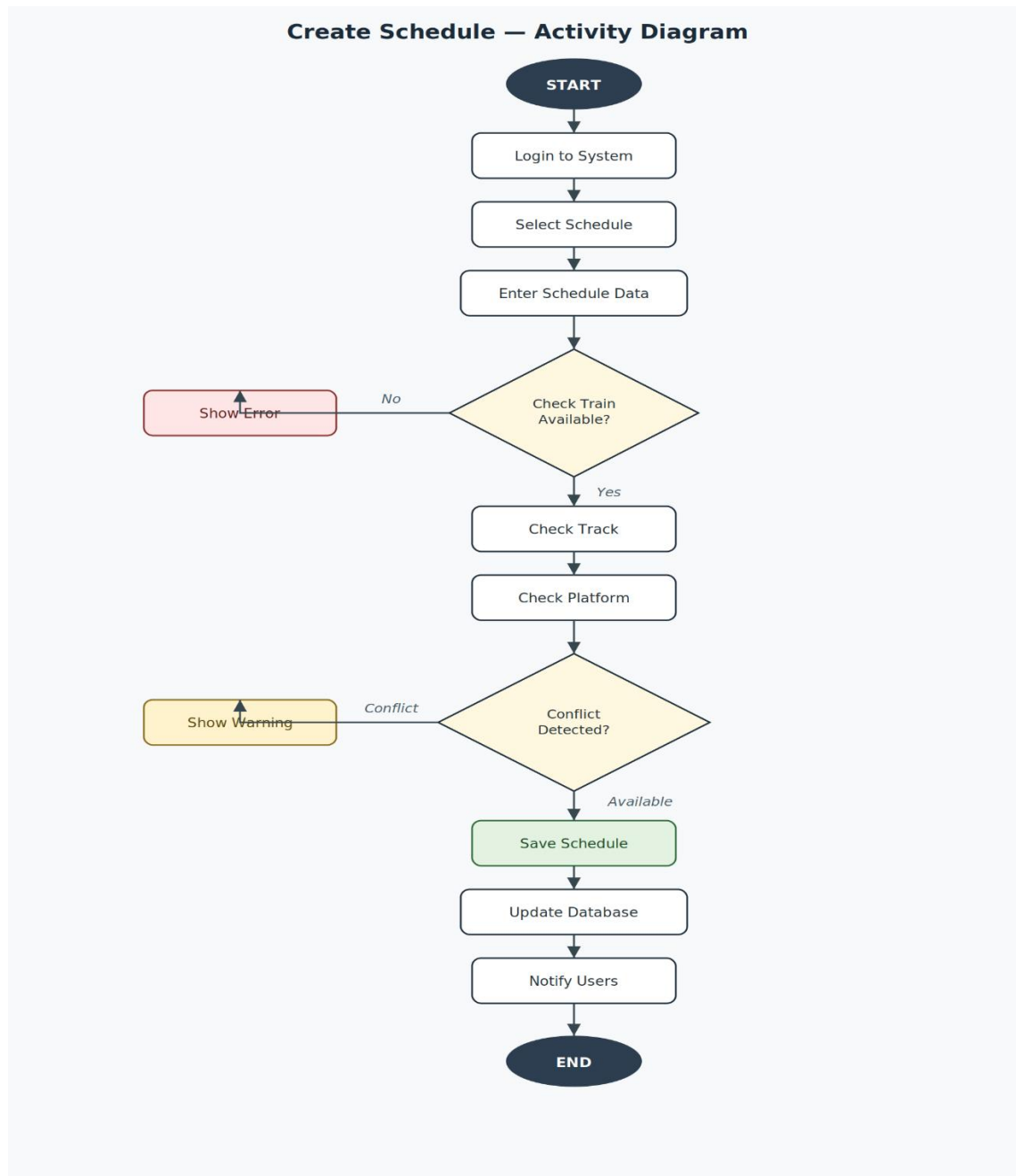
Train Scheduling System — Class Diagram



➤ Sequence Diagram:

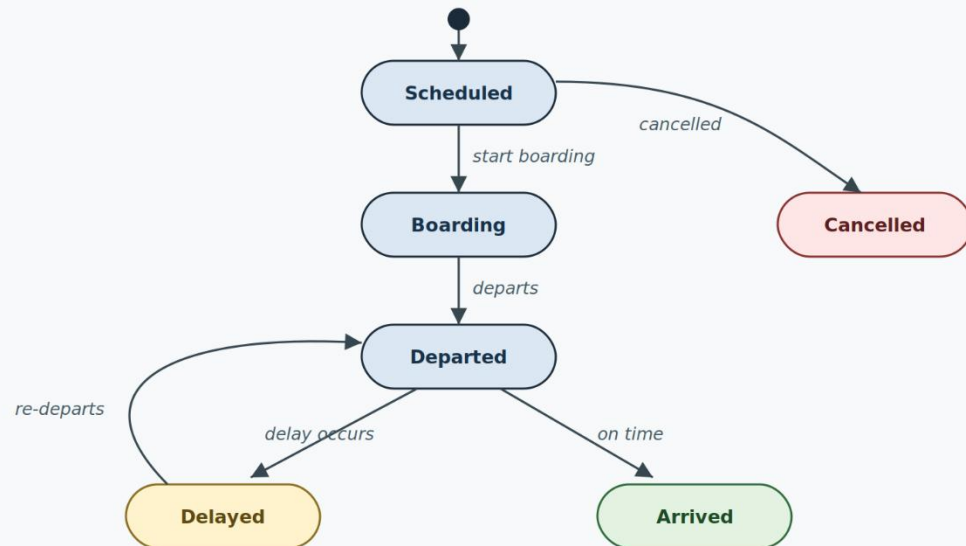


➤ Activity Diagram:



➤ State Diagram:

Train Journey — State Diagram

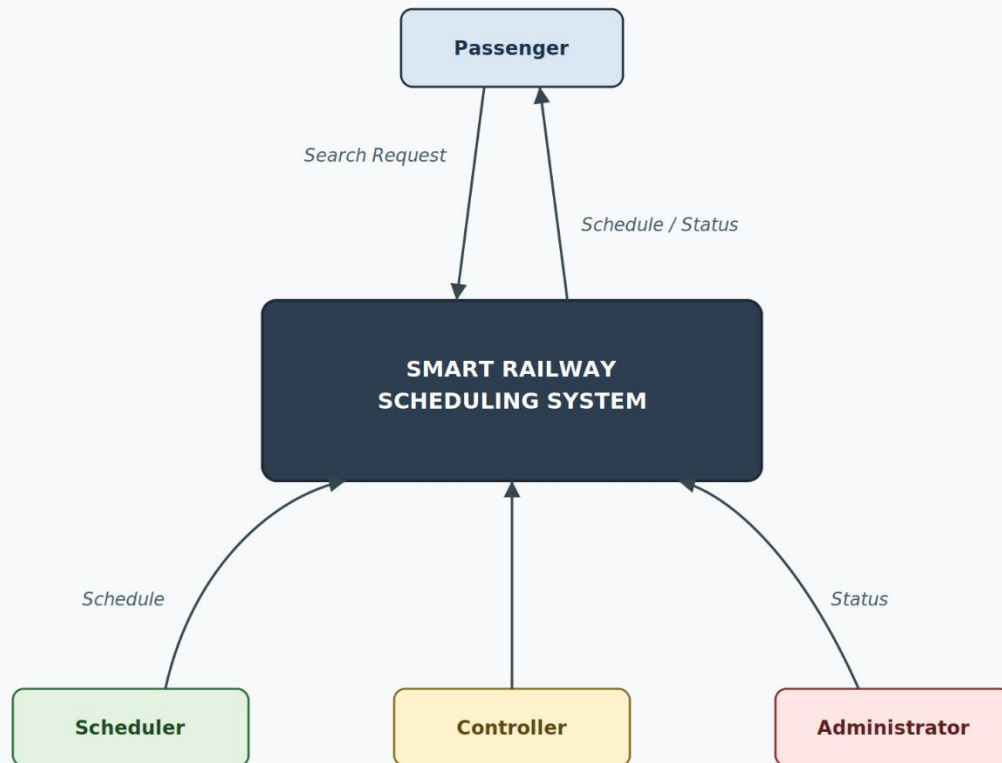


Possible states:

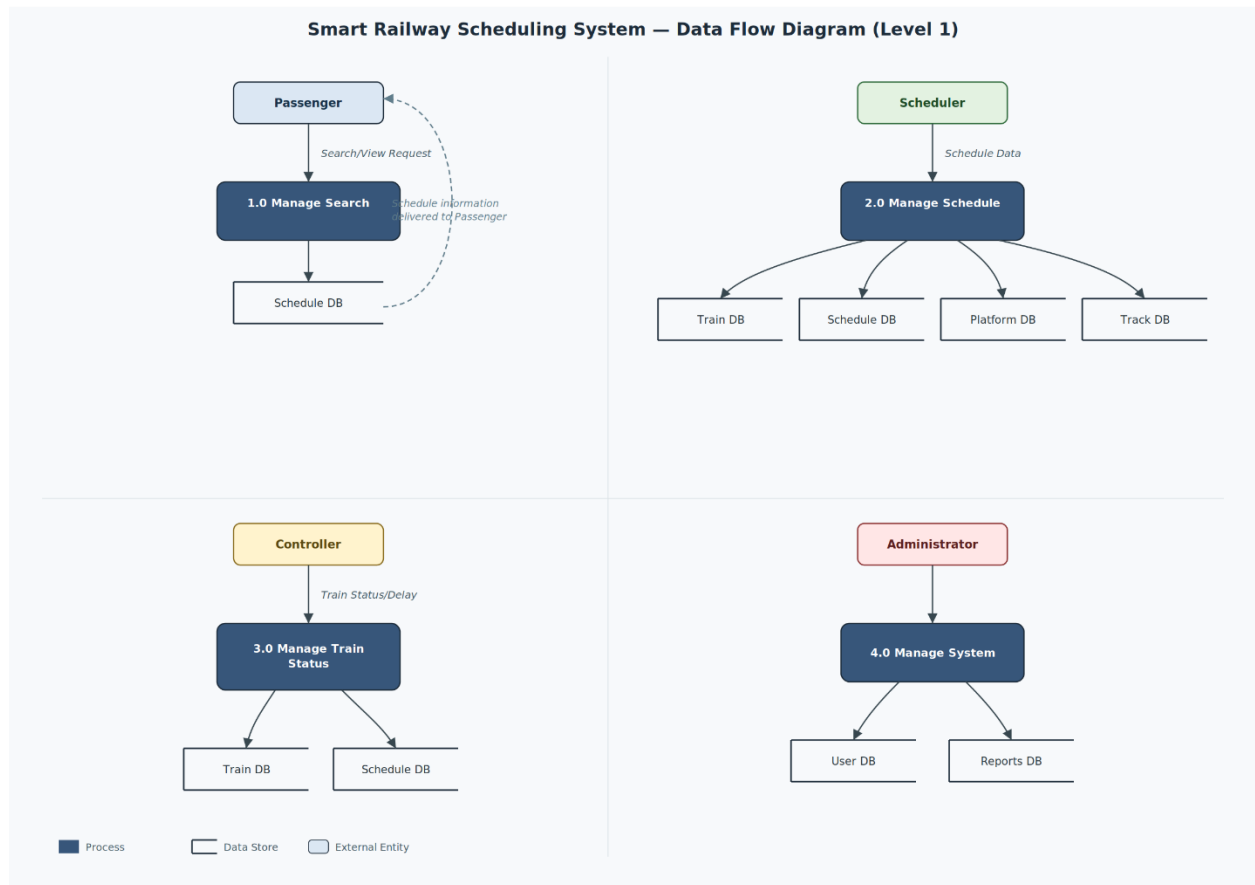
- Scheduled
- Boarding
- Departed
- Delayed
- Arrived
- Cancelled

➤ **Context Diagram:**

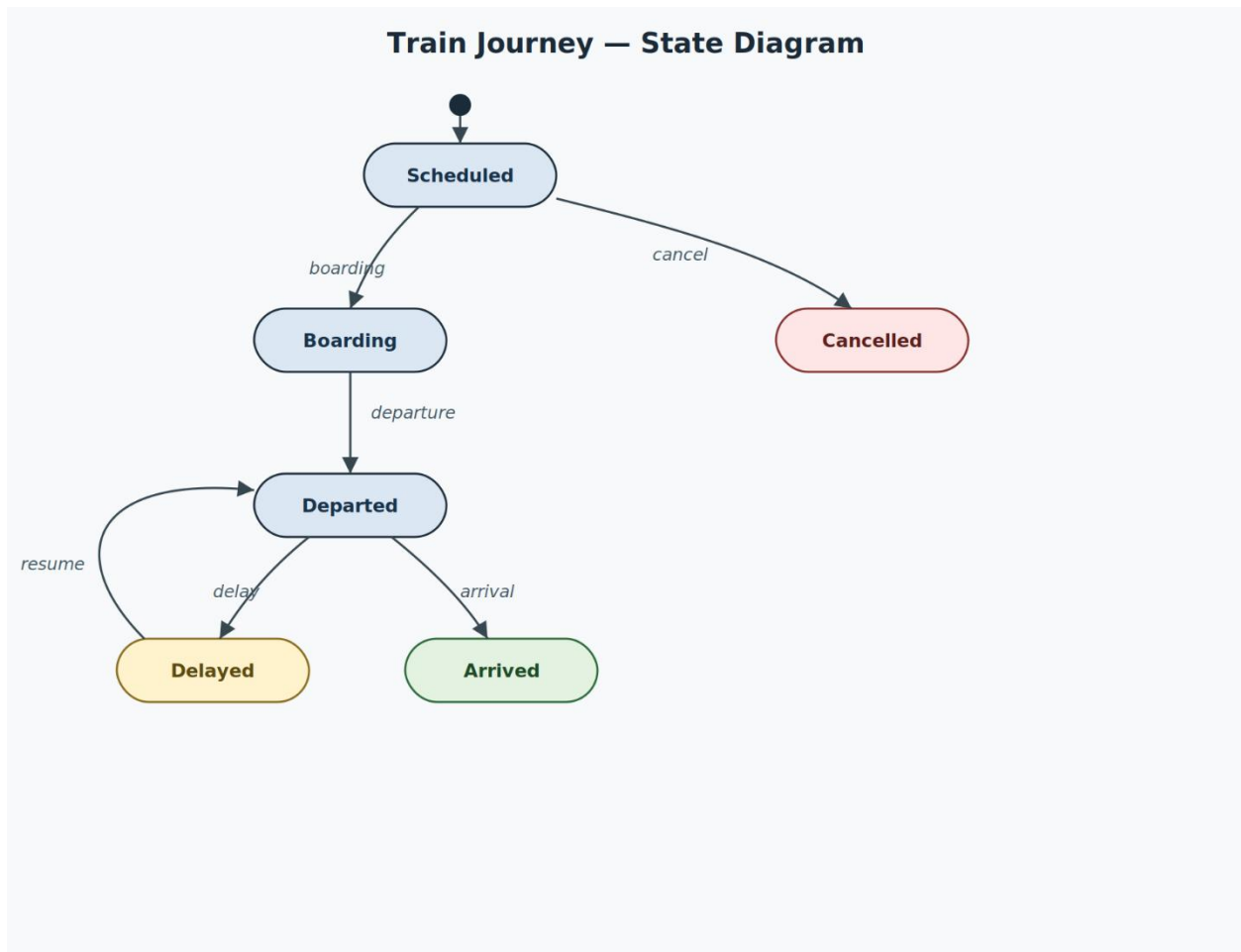
Smart Railway Scheduling System — Context Diagram



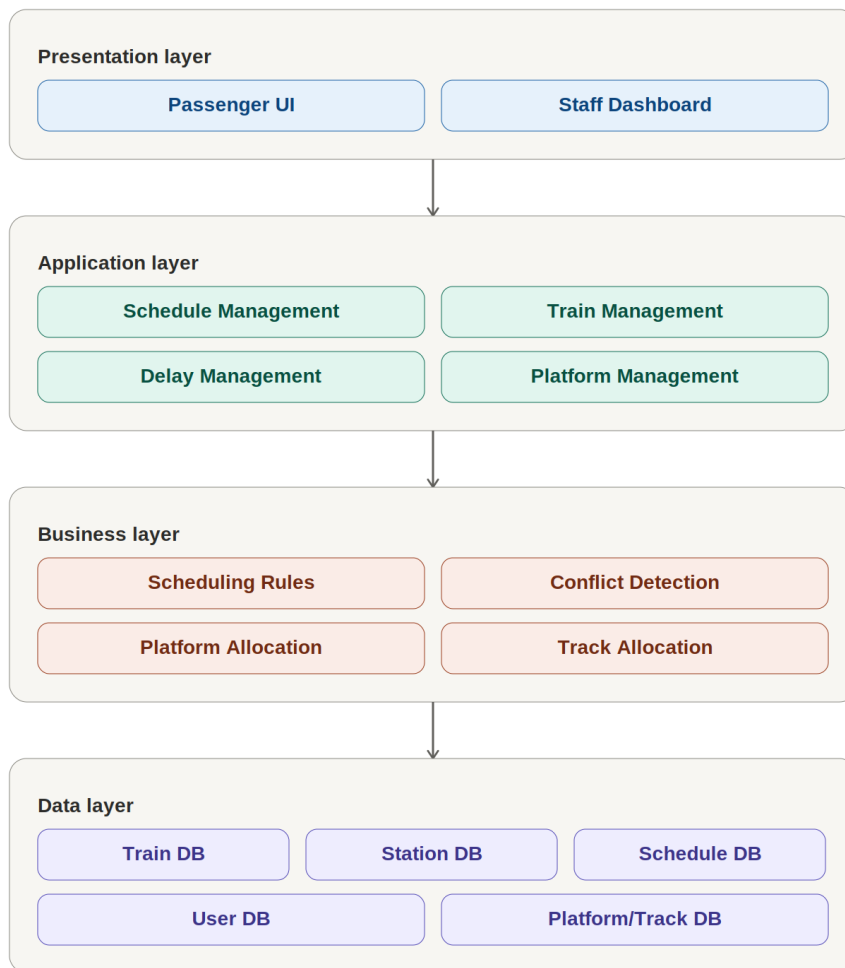
➤ Data Flow Diagram:



➤ **State Machine Diagram:**



➤ **Architecture Model:**



Presentation Layer:

Provides interfaces for passengers and railway employees.

Application Layer:

Handles system operations such as schedule creation and train management.

Business Layer:

Contains railway scheduling rules and conflict detection.

Data Layer:

Stores system information in databases.

➤ Testing Plan:

The assignment specifically requires a test plan covering unit, integration, and system testing, along with sample test cases.

Unit testing:

Individual modules are tested separately.

Examples:

- Login module
- Train module
- Schedule module
- Platform allocation module
- Delay module

Integration Testing:

Multiple modules are tested together.

Examples:

- Schedule + Train database
- Schedule + Platform allocation
- Delay + Notification
- Login + User management

System Testing:

The complete system is tested as one application.

Example:

Passenger searches for a train → system retrieves schedule → displays train
→ shows platform → displays delay if applicable.

➤ Sample Test cases:

Smart Railway Scheduling System – Test Cases			
Test ID	Test Case	Input	Expected Result
TC-01	Valid Login	Correct username/password	User successfully logs in
TC-02	Invalid Login	Wrong password	Error message displayed
TC-03	Search Train	Islamabad → Lahore	Available trains displayed
TC-04	Create Schedule	Valid train/time/platform	Schedule created
TC-05	Schedule Conflict	Same platform/time	Conflict warning displayed
TC-06	Update Status	Train → Delayed	Status changes to Delayed
TC-07	Invalid Train	Non-existing Train ID	Error displayed
TC-08	Platform Allocation	Available platform	Platform assigned
TC-09	Unavailable Platform	Occupied platform	Assignment rejected
TC-10	Cancel Schedule	Existing schedule	Schedule marked Cancelled
TC-11	Delay Notification	30-minute delay	Notification generated
TC-12	Unauthorized Update	Passenger attempts update	Access denied

➤ Demonstration (Version Control):

As required by the assignment, the group has used a version control system (GitHub) to host and track all documentation, diagrams, and files developed for this case study.

Repository Link:

<https://github.com/AbdulRehmanKhan101/SRSS-SE-Assignment>

Repository Contents:

- Final assignment document (Word/PDF)
- Diagram source files (use case, class, sequence, activity, state, DFD, context, architecture)
- README file describing the case study and project structure
- Commit history showing individual contributions from all four group members

➤ Project Planning Report:

This section presents the project planning aspects for the Smart Railway Scheduling System (SRSS) documentation project, including team roles, the project schedule, and a risk analysis.

Team Roles and Responsibilities:

Team Member	Role	Key Responsibilities
Mian Abubakar	Project Manager / Requirements Analyst	Coordinated the group, gathered and documented functional & non-functional requirements, problem statement and scope
Mirza Uzair Baig	System Modeler	Designed the use case, class, sequence, activity and state diagrams
Abdul Rehman Khan	Test Engineer / Quality Analyst	Prepared the test plan, sample test cases, and compiled the final documentation
Qasver Ali	Software Architect / Designer	Prepared the context diagram, DFD, state machine diagram and architecture model

Project Schedule (Gantt Chart):

The table below shows the estimated schedule for the phases of this documentation project, spread across six weeks leading up to submission.

Project Phase	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6
Requirement Gathering & Analysis						

System Modeling (Use Case, Class, Sequence, Activity, State)						
Design & Architecture (Context, DFD, Architecture Model)						
Test Planning & Test Case Design						
Documentation, Review & Submission						

Risk Analysis:

Risk	Likelihood	Impact	Mitigation Strategy
Incomplete or ambiguous information about the case study system	Medium	High	Cross-verify features using multiple sources and clarify with the instructor where needed
Team coordination and scheduling conflicts among group members	Medium	Medium	Hold regular meetings and clearly divide tasks with internal deadlines
Inconsistent diagram notation (UML) across group members' work	Medium	Medium	Use a single tool/template and cross-review all diagrams before merging
Last-minute changes/rework due to deadline pressure	High	High	Start each milestone early and track progress against internal milestones
Loss of work or version conflicts	Low	High	Use GitHub for version control with regular commits from all members